PASO 6. Seis Preguntas del Examen H3

1. Programación Orientada a Objetos (POO)

Pregunta:

¿Por qué el sistema NEXUS utiliza programación orientada a objetos en lugar de escribir todo el programa en un solo método?

Código de referencia:

```
Cadete cadete = new Cadete("Alex");
cadete.GastarEnergia(10);
```

Respuesta:

NEXUS utiliza POO porque organiza el programa en clases que tienen diferentes responsabilidades. Por ejemplo, `Cadete` administra los datos del jugador y su energía, mientras que `Inventario` administra los objetos. Esto permite organizar mejor el código, reutilizarlo y facilitar las modificaciones.

¿Por qué funciona el código?

Se crea un objeto llamado `cadete` a partir de la clase `Cadete`. Después, se llama al método `GastarEnergia(10)`, que reduce su energía en 10 puntos. Si comienza con 60, termina con 50.

2. Clases y Objetos

Pregunta:

¿Cuál es la diferencia entre una clase y un objeto? ¿Qué resultado produce el siguiente código?

Código de referencia:

```
Cadete cadete = new Cadete("Alex");
cadete.GastarEnergia(10);

Console.WriteLine(cadete.Energia);
```

Respuesta:

Una clase es un molde que define las características y acciones de algo; un objeto es una instancia creada a partir de esa clase. En este ejemplo, `Cadete` es la clase y `cadete` es el objeto.

Resultado esperado:

50

¿Por qué?

La energía inicial del cadete es 60. El método `GastarEnergia(10)` resta 10 puntos, por lo que queda con 50. La propiedad `Energia` permite consultar el valor actualizado.

3. Herencia y Polimorfismo

Pregunta:

En NEXUS, ¿qué sucede cuando se llama al método `Actuar()` de un objeto `Robot` o `Enemigo` a través de una variable de tipo `Personaje`?

Código de referencia:

```
private void Encuentro(Personaje personaje)
{
   personaje.Actuar(cadete);
}
```

Imaginemos que el método recibe un objeto `Robot`.

Respuesta:

Se ejecuta la versión de `Actuar()` que corresponde al objeto real, en este caso, `Robot.Actuar()`. Si recibe un `Enemigo`, se ejecuta `Enemigo.Actuar()`.

¿Por qué funciona?

Porque `Robot` y `Enemigo` heredan de la clase abstracta `Personaje` y pueden sobrescribir sus métodos. Esto es polimorfismo: una misma llamada puede producir comportamientos diferentes según el tipo real del objeto.

> **Nota:** Según el código revisado, el robot causa 10 puntos de daño y consume 5 puntos de energía del cadete, mientras que el enemigo causa 15 puntos de daño.

4. Interfaces

Pregunta:

¿Cómo podría NEXUS permitir que diferentes objetos, como terminales y puertas, tengan una acción de interacción sin necesidad de heredar de la misma clase concreta?

Ejemplo complementario (no implementado en el NEXUS revisado):

```
interface IInteractuable
{
    void Interactuar(Cadete cadete);
}

class Terminal : IInteractuable
{
    public void Interactuar(Cadete cadete)
    {
        cadete.GanarExperiencia(5);
    }
}
```

Respuesta:

Se puede utilizar una interfaz llamada `IInteractuable`. Esta define el método `Interactuar()`, que las clases que implementen la interfaz deben proporcionar.

¿Por qué funciona?

Porque `Terminal` implementa `IInteractuable` y, por eso, debe definir el método `Interactuar()`. Cuando se llama a ese método, el cadete gana 5 puntos de experiencia. Otra clase, como `Puerta`, podría implementar la misma interfaz con un comportamiento diferente.

La ventaja es que el sistema puede trabajar con objetos interactuables sin depender de una única clase concreta.

5. Listas y LINQ

Pregunta:

Si el inventario contiene los objetos `Celula`, `Escudo` y `Bateria`, con poderes de 25, 20 y 10 respectivamente, ¿qué nombres devuelve esta consulta?

Ejemplo complementario (LINQ no implementado en el NEXUS revisado):

```
using System.Linq;

var nombres = items
    .Where(o => o.Poder >= 20)
    .Select(o => o.Nombre);
```

Resultado esperado:

Celula
Escudo

¿Por qué funciona?

- `Where(o => o.Poder >= 20)` selecciona los objetos cuyo poder es mayor o igual a 20.
- `Select(o => o.Nombre)` obtiene solamente sus nombres.
- `Bateria` queda fuera porque su poder es 10.

> En NEXUS, el inventario sí utiliza una lista `List<Objeto>`, pero la consulta LINQ anterior es un ejemplo complementario para estudiar el tema.

6. Análisis y Corrección de Código

Pregunta:

Un cadete comienza con 60 puntos de energía, entra en la zona *Anomalía* y decide escapar del encuentro. ¿Con cuánta energía termina y qué ocurre con los objetivos de la misión?

Respuesta:

El cadete termina con 40 puntos de energía, porque gasta 10 puntos al entrar en la zona y otros 10 al escapar.

Además, según el comportamiento descrito en el código revisado, el método `ExplorarAnomalia()` marca como completados los objetivos relacionados con el núcleo y la supervivencia si el cadete continúa vivo, aunque no haya derrotado al enemigo.

¿Qué se debería corregir?

Si la misión exige derrotar al enemigo para completar el objetivo del núcleo, el programa debe comprobar que el enemigo haya sido derrotado antes de marcar ese objetivo como completado.

Propuesta de corrección conceptual:

```
if (enemigoDerrotado && cadete.Vida > 0)
{
    // Marcar el objetivo del núcleo como completado.
}
```

Este fragmento es una propuesta de corrección; los nombres de las variables deben adaptarse al código real.